

Consultas JPQL y transacciones

8. Consultas JPQL y transacciones

Lo que el naming y las Specifications no resuelven

Un ranking de editoriales, ordenado por cuántos libros tiene cada una en el catálogo. Prueba a resolverlo con lo que ya conoces.

Naming de método

No hay combinación de palabras que pida «agrupa por editorial y ordena por el número de libros de cada grupo». El naming deriva condiciones sobre campos, no agregaciones sobre una relación.

Specifications

Están pensadas para construir condiciones WHERE dinámicas, no un GROUP BY con ORDER BY sobre una agregación. Con CriteriaBuilder saldría una consulta rígida y más difícil de leer que el propio JPQL.

Aquí entra @Query con JPQL

Consultas fijas, siempre con la misma estructura, demasiado elaboradas para el nombre de un método o para combinar Specifications.

Las tres vías de consulta de Spring Data



Con @Query no deriva nada del nombre: usa literalmente el JPQL que le das. El nombre del método es solo una etiqueta descriptiva y la consulta real vive entera dentro de la anotación.

Naming de método

findByTitulo, findByEditorialId: simples, fijas, sin lógica adicional.

Specifications

Filtros dinámicos, combinables en tiempo de ejecución.

@Query con JPQL

Complejas y fijas: típicamente, agregaciones.

Qué es JPQL

El lenguaje de consultas de JPA. En vez de preguntar a la base de datos por sus tablas y columnas, preguntas a JPA por tus clases Java y sus propiedades.

Lo más simple: sin relaciones de por medio

SQL

```
SELECT * FROM libro
WHERE precio > 20
```

JPQL

```
SELECT l FROM Libro l
WHERE l.precio > 20
```

La diferencia se nota al cruzar una relación

SQL

```
SELECT e.* FROM editorial e
JOIN libro l ON l.editorial_id = e.id
```

JPQL

```
SELECT e FROM Editorial e
JOIN e.libros l
```

Solo cambian dos cosas

Libro (la clase, no la tabla) y l.precio (la propiedad del objeto). El resto se escribe igual.

e.libros navega la relación

JPA ya conoce la @OneToMany que declaraste: no repites la condición ON.

@Query con parámetros

Libros de una editorial, buscando por su nombre

```
public interface LibroRepository extends JpaRepository<Libro, Long> {  
  
    @Query("SELECT l FROM Libro l WHERE l.editorial.nombre = :nombreEditorial")  
    List<Libro> buscarPorNombreDeEditorial(  
        @Param("nombreEditorial") String nombreEditorial);  
}
```

l.editorial.nombre

Dos pasos de navegación con propiedades Java: primero la relación editorial, luego su nombre. Nunca columnas.

@Param("nombreEditorial")

Conecta el :nombreEditorial de la consulta con el parámetro del método. El texto debe coincidir exactamente.

Se valida al arrancar, no al ejecutar

Con l.editorial.nombre o :nombreEditorial, Spring falla al arrancar la aplicación. Una Specification, en cambio, solo descubre un nombre equivocado cuando alguien la ejecuta con una petición real.

Ejemplo completo: el ranking de editoriales

Agregación: resume varias filas en un resultado (COUNT, SUM, AVG), normalmente con GROUP BY. Aquí, JOIN + agregación + ordenar por ella: el caso típico de @Query.

```
public interface EditorialRepository
    extends JpaRepository<Editorial, Long> {

    @Query("""
        SELECT e
        FROM Editorial e
        LEFT JOIN e.libros l
        GROUP BY e
        ORDER BY COUNT(l) DESC
        """)
    List<Editorial> rankingPorNumeroDeLibros();
}
```



Para cada editorial...

cuenta sus libros y ordénalas de la que más tiene a la que menos.

La consulta, fragmento a fragmento

Fragmento	Qué hace
SELECT e	Devuelve el objeto Editorial completo, no una columna suelta.
FROM Editorial e	La entidad de partida, con el alias e para el resto de la consulta.
LEFT JOIN e.libros l	Navega la relación Editorial → libros y mantiene las editoriales sin ningún libro.
GROUP BY e	Agrupar, por cada editorial, los libros que comparten esa editorial.
ORDER BY COUNT(l) DESC	Ordena los grupos por cuántos libros tiene cada uno, de mayor a menor.

Por qué LEFT JOIN y no un JOIN normal: también aparecen las editoriales sin libros

5 libros

COUNT(l) = 5

2 libros

COUNT(l) = 2

Sin libros

COUNT(l) = 0: al final del ranking

Cuando necesitas SQL: funciones de ventana

Hay funcionalidad que no existe en JPQL. Las funciones de ventana de SQL son el ejemplo claro; la más simple es ROW_NUMBER(), que numera cada fila según el orden que le indique.

```
SELECT titulo, precio,  
       ROW_NUMBER() OVER  
         (ORDER BY precio DESC) AS posicion  
FROM libro;
```

OVER (ORDER BY ...) es la «ventana»

No agrupa nada: sigues teniendo una fila por libro, pero cada una lleva su posición.

Libros de partida

titulo	precio
El nombre del viento	24.99
Palabras de fuego	22.50
La sombra del viento	19.99
1984	12.99

Con la columna posicion añadida

titulo	precio	pos.
El nombre del viento	24.99	1
Palabras de fuego	22.50	2
La sombra del viento	19.99	3
1984	12.99	4

El ranking con posición: nativeQuery = true

Quieres que el propio ranking devuelva la posición (1º, 2º, 3º...) sin calcularla tú en Java. La ventana numera ahora los grupos del GROUP BY: eso ya es SQL de verdad, no JPQL.

```
@Query(value = """
    SELECT e.*, ROW_NUMBER() OVER (ORDER BY COUNT(l.id) DESC) AS posicion
    FROM editorial e LEFT JOIN libro l ON l.editorial_id = e.id
    GROUP BY e.id
    """, nativeQuery = true)
List<Object[]> rankingConPosicion();
```

Pierdes independencia de motor

nativeQuery = true: el texto es SQL tal cual lo entiende tu gestor, con sus funciones específicas. Solo vale donde exista esa sintaxis.

El resultado ya no son entidades

Pasa a ser List<Object[]>, una por fila: el motor no sabe mapear una columna calculada como posicion a ningún campo de tu entidad.

Escribir con JPQL: @Modifying

JPQL también ejecuta un UPDATE o un DELETE sobre varias filas sin cargarlas antes en Java una a una. Aquí, un descuento a todos los libros de una editorial.

```
@Modifying  
@Transactional  
@Query("UPDATE Libro l SET l.precio = l.precio * 0.9"  
        + " WHERE l.editorial.id = :editorialId")  
int aplicarDescuento(@Param("editorialId") Long editorialId);
```

@Modifying

Le dice a Spring Data que esta consulta no es un SELECT, sino que modifica datos.

El int que devuelve

El número de filas actualizadas: si el descuento afecta a cinco libros, devuelve 5. Ninguna se convierte antes en un objeto Libro.

Dónde escribes la operación, y qué transacción necesita

Como el procedimiento almacenado, este método evita recuperar todos los libros y recorrerlos con un bucle. Cambia dónde escribes la operación.

Procedimiento almacenado

La operación se guarda dentro de PostgreSQL y la aplicación la invoca.

JPQL con @Modifying

La actualización se declara en el repository de la aplicación. Es la que usarás aquí.

Aquí @Transactional no puede ser readOnly

Un UPDATE o un DELETE modifica datos y necesita una transacción de escritura: @Transactional a secas, igual que en los métodos create() y update() de tus servicios.

Ideas clave

1

JPQL opera sobre entidades y propiedades Java, no sobre tablas y columnas SQL.

2

Tres vías: naming (simple y fijo), Specifications (dinámico) y `@Query/JPQL` (complejo y fijo, típicamente agregaciones).

3

`@Param` conecta un `:parámetro` del JPQL con un argumento del método: el nombre debe coincidir exactamente.

4

Spring comprueba las `@Query` al arrancar; en una Specification, algunos errores solo aparecen al ejecutar.

5

Una agregación (COUNT, SUM, AVG con GROUP BY) resume varias filas. LEFT JOIN conserva los grupos vacíos.

6

`nativeQuery = true` escribe SQL literal: pierdes independencia de motor y el resultado pasa a `Object[]` por fila.

7

`@Modifying` marca una `@Query` como escritura (UPDATE/DELETE) y devuelve las filas afectadas, no una lista.

8

Necesita `@Transactional` a secas, nunca `readOnly`.